Problem 1 Semantics

Consider a variant of the IMP language which features *local* variables and *single-argument* procedures.

The abstract syntax for this language is shown below where $z \in \mathbb{Z}$ refers to integer constants and $x \in \mathsf{Var}$ refers to variables:

$$\begin{array}{rl} \mathsf{Expressions} & e ::= z \mid x \mid e_1 + e_2 \\ \mathsf{Commands} & c ::= \mathsf{var}\ x := e \mid x := e \mid c_1; c_2 \mid \mathsf{call}\ f(x) \\ \mathsf{Programs} & P ::= \mathsf{def}\ f(x)\ \{c_f\}\ ;\ \ldots \\ & \qquad ;\ \mathsf{def}\ g(y)\{c_g\}\ ;\ c_{\mathsf{main}} \end{array}$$

We define the semantics for our modified IMP language in terms of evaluation relations over *states* and *abstract syntax*. For program states, we refer to:

- **Variable environments** map names to memory locations.

$$\rho : \mathsf{Var} \to \mathsf{Addr}$$

- **Heaps** stores integer values at memory locations.

$$h : \mathsf{Addr} \to \mathbb{Z}$$

- **Procedure environments** give, for each procedure f, its parameter and command body.

$$\Pi(f) = (x, c_f)$$

Our semantics uses the following evaluation relations:

- **Expression evaluation** — $\langle e, \rho, h\rangle \to z$.
- **Command evaluation** — $\langle c, \rho, h\rangle \to \langle \rho, h'\rangle$

Functional update notation $\rho[x : a]$ and $h[a : v]$ are used to create new environments and heaps.

Assume $\mathit{fresh}(h)$ returns a fresh heap location *not* in the domain of h.

Here are example inference rules which describe the variable expressions, variable declarations, and variable assignment.

$$\frac{}{\langle x, \rho, h\rangle \to h(\rho(x))}\ \textsc{Var} \qquad \frac{\langle e, \rho, h\rangle \to z}{\langle x := e, \rho, h\rangle \to \langle \rho, h[\rho(x) : z]\rangle}\ \textsc{Assign}$$

$$\frac{a = \mathit{fresh}(h) \qquad \langle e, \rho, h\rangle \to z \qquad \rho' = \rho[x : a] \qquad h' = h[a : z]}{\langle \mathsf{var}\ x := e, \rho, h\rangle \to \langle \rho', h'\rangle}\ \textsc{Decl}$$

We will compare two different evaluation strategies for IMP that appear in many popular programming languages: *call-by-value* and *call-by-reference*. We define each as follows:

- *Call-by-value* — The execution of a procedure call receives a copy of the argument's value.
- *Call-by-reference* — The execution of a procedure call receives a reference to the argument's original memory location.

Consider the following program fragment where initially $\rho(x) = a_x$ and $h(a_x) = 0$:

```
def inc(y) {
    y := y + 1
};

x := 0;
call inc(x);
call inc(x);
```

Part a

Under *call-by-value* semantics, what is the final value stored at $h(a_x)$ after executing the whole fragment? Briefly justify your answer in English.

Part b

Under *call-by-reference* semantics, what is the final value stored at $h(a_x)$ after executing the whole fragment? Briefly justify your answer in English.

Part c

Extend the command evaluation relation $\langle c, \rho, h\rangle \to \langle \rho', h'\rangle$ with one or more inference rules describing procedure calls under *call-by-value* semantics.

Your solution should include an inference rule following the template below.

$$\frac{\mathsf{premise}_1 \qquad \ldots \qquad \mathsf{premise}_n}{\langle \mathsf{call}\ f(y), \rho, h\rangle \to \langle \rho_{\mathsf{result}}, h_{\mathsf{result}}\rangle}$$

Part d

Extend the command evaluation relation $\langle c, \rho, h\rangle \to \langle \rho', h'\rangle$ with one or more inference rules describing procedure calls under *call-by-reference* semantics.

Your solution should include an inference rule following the template below.

$$\frac{\mathsf{premise}_1 \qquad \ldots \qquad \mathsf{premise}_n}{\langle \mathsf{call}\ f(y), \rho, h\rangle \to \langle \rho_{\mathsf{result}}, h_{\mathsf{result}}\rangle}$$